Experiment No. 1

Code:

```python
# Experiment 1: Library Management System using OOP
from datetime import date, timedelta


class Book:
    total_books = 0                      # class (static) attribute

    def __init__(self, book_id, title, author):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.is_available = True
        Book.total_books += 1

    def __str__(self):
        status = "Available" if self.is_available else "Issued"
        return f"[{self.book_id}] {self.title} by {self.author} - {status}"


class Patron:
    MAX_BOOKS = 2

    def __init__(self, patron_id, name):
        self.patron_id = patron_id
        self.name = name
        self.borrowed = {}               # book_id -> due date

    def __str__(self):
        return f"Patron {self.patron_id}: {self.name} (borrowed {len(self.borrowed)})"


class Library:
    LOAN_DAYS = 14
    FINE_PER_DAY = 2

    def __init__(self, name):
        self.name = name
        self.books = {}
        self.patrons = {}

    def add_book(self, book):
        self.books[book.book_id] = book
        print(f"Added book    : {book.title}")

    def register_patron(self, patron):
        self.patrons[patron.patron_id] = patron
        print(f"Registered    : {patron.name}")

    def borrow_book(self, patron_id, book_id, today):
        patron, book = self.patrons.get(patron_id), self.books.get(book_id)
        if not patron or not book:
            print("Error: invalid patron or book id"); return
        if not book.is_available:
            print(f"Sorry, '{book.title}' is already issued."); return
        if len(patron.borrowed) >= Patron.MAX_BOOKS:
            print(f"{patron.name} reached the limit of {Patron.MAX_BOOKS} books."); return
        book.is_available = False
        patron.borrowed[book_id] = today + timedelta(days=self.LOAN_DAYS)
        print(f"{patron.name} borrowed '{book.title}' (due {patron.borrowed[book_id]})")

    def return_book(self, patron_id, book_id, today):
        patron, book = self.patrons[patron_id], self.books[book_id]
        if book_id not in patron.borrowed:
            print("This book was not borrowed by the patron."); return
        due = patron.borrowed.pop(book_id)
        book.is_available = True
        late = max((today - due).days, 0)
        msg = f"{patron.name} returned '{book.title}'"
        msg += f" - late by {late} days, fine Rs.{late * self.FINE_PER_DAY}" if late else " on time."
        print(msg)

    def show_books(self):
        print(f"\n--- Books in {self.name} ---")
        for b in self.books.values():
            print(b)

```

```
77
78  lib = Library("MIT-ADT Central Library")
79  lib.add_book(Book(101, "Fluent Python", "Luciano Ramalho"))
80  lib.add_book(Book(102, "Python Cookbook", "David Beazley"))
81  lib.add_book(Book(103, "Python in a Nutshell", "Alex Martelli"))
82  lib.register_patron(Patron(1, "Yash"))
83  lib.register_patron(Patron(2, "Riya"))
84  print()
85  d = date(2026, 10, 1)
86  lib.borrow_book(1, 101, d)
87  lib.borrow_book(2, 101, d)             # already issued
88  lib.borrow_book(1, 102, d)
89  lib.borrow_book(1, 103, d)             # limit reached
90  lib.show_books()
91  print()
92  lib.return_book(1, 101, d + timedelta(days=20))   # late
93  lib.return_book(1, 102, d + timedelta(days=10))   # on time
94  lib.show_books()
95  print("\nTotal books created:", Book.total_books)
96  print(lib.patrons[1])
```

Output:

```
Added book    : Fluent Python
Added book    : Python Cookbook
Added book    : Python in a Nutshell
Registered    : Yash
Registered    : Riya
Yash borrowed 'Fluent Python' (due 2026-10-15)
Sorry, 'Fluent Python' is already issued.
Yash borrowed 'Python Cookbook' (due 2026-10-15)
Yash reached the limit of 2 books.
--- Books in MIT-ADT Central Library ---
[101] Fluent Python by Luciano Ramalho - Issued
[102] Python Cookbook by David Beazley - Issued
[103] Python in a Nutshell by Alex Martelli - Available
Yash returned 'Fluent Python' - late by 6 days, fine Rs.12
Yash returned 'Python Cookbook' on time.
--- Books in MIT-ADT Central Library ---
[101] Fluent Python by Luciano Ramalho - Available
[102] Python Cookbook by David Beazley - Available
[103] Python in a Nutshell by Alex Martelli - Available
Total books created: 3
Patron 1: Yash (borrowed 0)
```

Experiment No. 2

Code:

```python
# Experiment 2: Dynamic report generator - decorators, class methods, magic methods
import functools


def log_call(func):                         # function decorator
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"  [LOG] Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper


def uppercase(func):                        # formatting decorator
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs).upper()
    return wrapper


def bordered(func):                         # formatting decorator
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        text = func(*args, **kwargs)
        width = max(len(l) for l in text.splitlines()) + 4
        line = "+" + "-" * (width - 2) + "+"
        body = "\n".join("| " + l.ljust(width - 4) + " |" for l in text.splitlines())
        return f"{line}\n{body}\n{line}"
    return wrapper


class Report:
    templates = {}                          # class-level template registry

    def __init__(self, title, data, template="simple"):
        self.title, self.data, self.template = title, data, template

    @classmethod
    def register_template(cls, name, fmt):
        cls.templates[name] = fmt
        print(f"Template '{name}' registered")

    @classmethod
    def from_dict(cls, d, template="simple"):
        return cls(d["title"], d["data"], template)

    @log_call
    def render(self):
        fmt = self.templates[self.template]
        rows = "\n".join(fmt["row"].format(k=k, v=v) for k, v in self.data.items())
        return fmt["head"].format(title=self.title) + "\n" + rows

    @log_call
    @uppercase
    def render_upper(self):
        return self._plain()

    @log_call
    @bordered
    def render_boxed(self):
        return self._plain()

    def _plain(self):
        fmt = self.templates[self.template]
        rows = "\n".join(fmt["row"].format(k=k, v=v) for k, v in self.data.items())
        return fmt["head"].format(title=self.title) + "\n" + rows

    # ---- magic methods ----
    def __str__(self):
        return self._plain()

    def __repr__(self):
        return f"Report(title={self.title!r}, items={len(self.data)})"

    def __len__(self):
        return len(self.data)

```

```
    def __add__(self, other):
        return Report(f"{self.title} + {other.title}", {**self.data, **other.data}, self.template)

    def __call__(self, template):
        self.template = template
        return self


Report.register_template("simple", {"head": "== {title} ==", "row": "{k}: {v}"})
Report.register_template("dash", {"head": "--- {title} ---", "row": " * {k} -> {v}"})

r1 = Report("Sales Q1", {"Pune": 1200, "Mumbai": 1850})
r2 = Report.from_dict({"title": "Sales Q2", "data": {"Delhi": 990}})

print("\n1) Simple template:");      print(r1.render())
print("\n2) Dash template via __call__:"); print(r1("dash").render())
print("\n3) Uppercase decorator:");  print(r1.render_upper())
print("\n4) Bordered decorator:");   print(r1.render_boxed())
print("\n5) Magic methods:")
combined = r1 + r2
print("repr    :", repr(combined))
print("len     :", len(combined))
print("str     :\n" + str(combined))
```

Output:

```
Template 'simple' registered
Template 'dash' registered
1) Simple template:
  [LOG] Calling render
== Sales Q1 ==
Pune: 1200
Mumbai: 1850
2) Dash template via __call__:
  [LOG] Calling render
--- Sales Q1 ---
 * Pune -> 1200
 * Mumbai -> 1850
3) Uppercase decorator:
  [LOG] Calling render_upper
--- SALES Q1 ---
 * PUNE -> 1200
 * MUMBAI -> 1850
4) Bordered decorator:
  [LOG] Calling render_boxed
+------------------+
| --- Sales Q1 ---  |
|  * Pune -> 1200   |
|  * Mumbai -> 1850 |
+------------------+
5) Magic methods:
repr    : Report(title='Sales Q1 + Sales Q2', items=3)
len     : 3
str     :
--- Sales Q1 + Sales Q2 ---
 * Pune -> 1200
 * Mumbai -> 1850
 * Delhi -> 990
```

Experiment No. 3

Code:

```python
# Experiment 3: Configurable Payment Processing System - Strategy Pattern
from abc import ABC, abstractmethod


class PaymentStrategy(ABC):
    @abstractmethod
    def pay(self, amount): ...


class CreditCardPayment(PaymentStrategy):
    def __init__(self, number, holder):
        self.number, self.holder = number, holder

    def pay(self, amount):
        fee = round(amount * 0.02, 2)
        return f"Paid Rs.{amount} by Credit Card (**** {self.number[-4:]}, {self.holder}); fee Rs.{fee}"


class PayPalPayment(PaymentStrategy):
    def __init__(self, email):
        self.email = email

    def pay(self, amount):
        fee = round(amount * 0.03, 2)
        return f"Paid Rs.{amount} via PayPal ({self.email}); fee Rs.{fee}"


class BitcoinPayment(PaymentStrategy):
    RATE = 5_000_000                        # Rs. per BTC (demo value)

    def __init__(self, wallet):
        self.wallet = wallet

    def pay(self, amount):
        return f"Paid Rs.{amount} = {amount / self.RATE:.8f} BTC from wallet {self.wallet[:8]}..."


class UPIPayment(PaymentStrategy):
    def __init__(self, upi_id):
        self.upi_id = upi_id

    def pay(self, amount):
        return f"Paid Rs.{amount} via UPI ({self.upi_id}); no fee"


class PaymentProcessor:                     # context
    def __init__(self, strategy=None):
        self._strategy = strategy

    def set_strategy(self, strategy):       # strategy chosen at run time
        self._strategy = strategy

    def checkout(self, amount):
        if self._strategy is None:
            raise ValueError("No payment method selected")
        if amount <= 0:
            raise ValueError("Amount must be positive")
        return self._strategy.pay(amount)


p = PaymentProcessor()
print("Order 1:", end=" ")
p.set_strategy(CreditCardPayment("1234567812345678", "Yash"));   print(p.checkout(2500))
print("Order 2:", end=" ")
p.set_strategy(PayPalPayment("yash@example.com"));               print(p.checkout(1800))
print("Order 3:", end=" ")
p.set_strategy(BitcoinPayment("1A1zP1eP5QGefi2DMPTf"));          print(p.checkout(50000))
print("Order 4:", end=" ")
p.set_strategy(UPIPayment("yash@upi"));                          print(p.checkout(700))

print("\nConfigurable (choose by name at runtime):")
registry = {"card": lambda: CreditCardPayment("9999888877776666", "Riya"),
            "upi": lambda: UPIPayment("riya@upi")}
for choice in ["upi", "card"]:
    p.set_strategy(registry[choice]())
    print(f"  {choice:5s} ->", p.checkout(999))
```

```
77
78 try:
79     PaymentProcessor().checkout(100)
80 except ValueError as e:
81     print("\nError handled:", e)
```

Output:

```
Order 1: Paid Rs.2500 by Credit Card (**** 5678, Yash); fee Rs.50.0
Order 2: Paid Rs.1800 via PayPal (yash@example.com); fee Rs.54.0
Order 3: Paid Rs.50000 = 0.01000000 BTC from wallet 1A1zP1eP...
Order 4: Paid Rs.700 via UPI (yash@upi); no fee
Configurable (choose by name at runtime):
  upi   -> Paid Rs.999 via UPI (riya@upi); no fee
  card  -> Paid Rs.999 by Credit Card (**** 6666, Riya); fee Rs.19.98
Error handled: No payment method selected
```

Experiment No. 4

Code:

```python
# Experiment 4: nth Fibonacci number using Dynamic Programming
import time
from functools import lru_cache


def fib_recursive(n):                       # naive, exponential
    return n if n < 2 else fib_recursive(n - 1) + fib_recursive(n - 2)


def fib_memo(n, memo=None):                 # top-down (memoization)
    if memo is None:
        memo = {}
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]


def fib_tab(n):                             # bottom-up (tabulation)
    if n < 2:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]


def fib_iter(n):                            # space optimised O(1)
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a


def timeit(f, n):
    t = time.perf_counter(); r = f(n)
    return r, (time.perf_counter() - t) * 1000


print("First 15 terms:", [fib_tab(i) for i in range(15)])
print("\n{:<6}{:>12}{:>12}{:>12}{:>12}".format("n", "memo", "tabulation", "iterative", "naive"))
for n in (10, 20, 30):
    vals = {f.__name__: f(n) for f in (fib_memo, fib_tab, fib_iter, fib_recursive)}
    assert len(set(vals.values())) == 1
    print("{:<6}{:>12}{:>12}{:>12}{:>12}".format(n, *vals.values()))

print("\nTime taken (ms) for n = 32:")
for f in (fib_recursive, fib_memo, fib_tab, fib_iter):
    r, ms = timeit(f, 32)
    print(f"  {f.__name__:<14} result={r}  time={ms:.4f} ms")

print("\nLarge n using tabulation:")
print("  F(100) =", fib_tab(100))
print("  F(200) =", fib_iter(200))
```

Output:

```
First 15 terms: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
n             memo  tabulation   iterative       naive
10              55          55          55          55
20            6765        6765        6765        6765
30          832040      832040      832040      832040
Time taken (ms) for n = 32:
  fib_recursive  result=2178309  time=329.8729 ms
  fib_memo       result=2178309  time=0.0174 ms
  fib_tab        result=2178309  time=0.0110 ms
  fib_iter       result=2178309  time=0.0026 ms
Large n using tabulation:
  F(100) = 354224848179261915075
  F(200) = 280571172992510140037611932413038677189525
```

Experiment No. 5

Code:

```python
# Experiment 5: Longest Common Subsequence using Dynamic Programming


def lcs(X, Y):
    m, n = len(X), len(Y)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if X[i - 1] == Y[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    # backtrack to build the subsequence
    i, j, seq = m, n, []
    while i > 0 and j > 0:
        if X[i - 1] == Y[j - 1]:
            seq.append(X[i - 1]); i -= 1; j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return dp[m][n], list(reversed(seq)), dp


def show_table(X, Y, dp):
    print("      " + "  ".join(f"{c:>2}" for c in "-" + Y))
    for i, row in enumerate(dp):
        label = "-" if i == 0 else X[i - 1]
        print(f"  {label:>2}  " + "  ".join(f"{v:>2}" for v in row))


tests = [("AGGTAB", "GXTXAYB"), ("ABCBDAB", "BDCABA"), ("PYTHON", "PROGRAMMING"), ("ABC", "XYZ")]
for X, Y in tests:
    length, seq, dp = lcs(X, Y)
    print(f"X = {X!r}, Y = {Y!r}")
    print(f"   LCS length = {length}, LCS = {''.join(seq)!r}\n")

print("DP table for X='AGGTAB', Y='GXTXAYB':")
_, _, dp = lcs("AGGTAB", "GXTXAYB")
show_table("AGGTAB", "GXTXAYB", dp)

print("\nLCS of two integer lists:")
a, b = [1, 3, 4, 1, 2, 3], [3, 4, 1, 2, 1, 3]
l, s, _ = lcs(a, b)
print(f"  {a} and {b} -> length {l}, subsequence {s}")
```

Output:

```
X = 'AGGTAB', Y = 'GXTXAYB'
   LCS length = 4, LCS = 'GTAB'
X = 'ABCBDAB', Y = 'BDCABA'
   LCS length = 4, LCS = 'BCBA'
X = 'PYTHON', Y = 'PROGRAMMING'
   LCS length = 3, LCS = 'PON'
X = 'ABC', Y = 'XYZ'
   LCS length = 0, LCS = ''
DP table for X='AGGTAB', Y='GXTXAYB':
       -   G   X   T   X   A   Y   B
   -   0   0   0   0   0   0   0   0
   A   0   0   0   0   0   1   1   1
   G   0   1   1   1   1   1   1   1
   G   0   1   1   1   1   1   1   1
   T   0   1   1   2   2   2   2   2
   A   0   1   1   2   2   3   3   3
   B   0   1   1   2   2   3   3   4
LCS of two integer lists:
  [1, 3, 4, 1, 2, 3] and [3, 4, 1, 2, 1, 3] -> length 5, subsequence [3, 4, 1, 2, 3]
```

Experiment No. 6

Code:

```python
# Experiment 6: 0/1 Knapsack - bottom-up and top-down dynamic programming
import sys
sys.setrecursionlimit(10000)


def knapsack_topdown(W, wt, val, n, memo=None):
    if memo is None:
        memo = {}
    if n == 0 or W == 0:
        return 0
    if (n, W) in memo:
        return memo[(n, W)]
    if wt[n - 1] > W:
        res = knapsack_topdown(W, wt, val, n - 1, memo)
    else:
        res = max(val[n - 1] + knapsack_topdown(W - wt[n - 1], wt, val, n - 1, memo),
                  knapsack_topdown(W, wt, val, n - 1, memo))
    memo[(n, W)] = res
    return res


def knapsack_bottomup(W, wt, val):
    n = len(wt)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(W + 1):
            dp[i][w] = dp[i - 1][w]
            if wt[i - 1] <= w:
                dp[i][w] = max(dp[i][w], val[i - 1] + dp[i - 1][w - wt[i - 1]])
    chosen, w = [], W                       # trace back selected items
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            chosen.append(i - 1); w -= wt[i - 1]
    return dp[n][W], sorted(chosen), dp


# Examples from the lab manual
for W, profit, weight in [(4, [1, 2, 3], [4, 5, 1]), (3, [1, 2, 3], [4, 5, 6])]:
    td = knapsack_topdown(W, weight, profit, len(weight))
    bu, items, _ = knapsack_bottomup(W, weight, profit)
    print(f"W={W} profit={profit} weight={weight}")
    print(f"   top-down = {td}, bottom-up = {bu}, items (0-indexed) = {items}\n")

values = [60, 100, 120, 80, 30]
weights = [10, 20, 30, 15, 5]
W = 50
td = knapsack_topdown(W, weights, values, len(values))
bu, items, dp = knapsack_bottomup(W, weights, values)
print(f"Items (weight, value): {list(zip(weights, values))}, capacity = {W}")
print("Top-down  (memoization) maximum value :", td)
print("Bottom-up (tabulation)  maximum value :", bu)
print("Selected items (weight, value)        :", [(weights[i], values[i]) for i in items])
print("Total weight used                     :", sum(weights[i] for i in items))
print("Both methods agree                    :", td == bu)
```

Output:

```
W=4 profit=[1, 2, 3] weight=[4, 5, 1]
   top-down = 3, bottom-up = 3, items (0-indexed) = [2]
W=3 profit=[1, 2, 3] weight=[4, 5, 6]
   top-down = 0, bottom-up = 0, items (0-indexed) = []
Items (weight, value): [(10, 60), (20, 100), (30, 120), (15, 80), (5, 30)], capacity = 50
Top-down  (memoization) maximum value : 270
Bottom-up (tabulation)  maximum value : 270
Selected items (weight, value)        : [(10, 60), (20, 100), (15, 80), (5, 30)]
Total weight used                     : 50
Both methods agree                    : True
```

Experiment No. 7

Code:

```python
# Experiment 7: Find email addresses using regular expressions
import re

pattern = r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}\b"

text = """Contact us at support@mitadt.edu.in or admin@example.com.
Send resumes to hr.team@company-name.org, jobs_2026@startup.io
Invalid ones: user@@mail.com, @nodomain.com, name@site, plain.text.com
Personal: john.doe+news@gmail.com and Yash_Patil99@yahoo.co.in"""

print("Input text:\n" + text)
emails = re.findall(pattern, text)
print("\nPattern:", pattern)
print(f"\nValid email addresses found ({len(emails)}):")
for i, e in enumerate(emails, 1):
    user, domain = e.split("@")
    print(f"  {i}. {e:<30} username={user:<16} domain={domain}")

print("\nValidation of individual strings:")
for s in ["abc@xyz.com", "bad@@x.com", "no_at_sign.com", "a.b-c@sub.domain.org", "x@y"]:
    ok = re.fullmatch(pattern, s) is not None
    print(f"  {s:<25} -> {'VALID' if ok else 'INVALID'}")

print("\nMasking emails with re.sub:")
print(re.sub(pattern, "[hidden]", "Mail yash@gmail.com or riya@mit.edu today"))
```

Output:

```
Input text:
Contact us at support@mitadt.edu.in or admin@example.com.
Send resumes to hr.team@company-name.org, jobs_2026@startup.io
Invalid ones: user@@mail.com, @nodomain.com, name@site, plain.text.com
Personal: john.doe+news@gmail.com and Yash_Patil99@yahoo.co.in
Pattern: \b[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}\b
Valid email addresses found (6):
  1. support@mitadt.edu.in          username=support          domain=mitadt.edu.in
  2. admin@example.com              username=admin            domain=example.com
  3. hr.team@company-name.org       username=hr.team          domain=company-name.org
  4. jobs_2026@startup.io           username=jobs_2026        domain=startup.io
  5. john.doe+news@gmail.com        username=john.doe+news    domain=gmail.com
  6. Yash_Patil99@yahoo.co.in       username=Yash_Patil99     domain=yahoo.co.in
Validation of individual strings:
  abc@xyz.com               -> VALID
  bad@@x.com                -> INVALID
  no_at_sign.com            -> INVALID
  a.b-c@sub.domain.org      -> VALID
  x@y                       -> INVALID
Masking emails with re.sub:
Mail [hidden] or [hidden] today
```

Experiment No. 8

Code:

```python
# Experiment 8: File handling - count lines, extract first two lines, write to new file
lines = [
    "Python is a high-level programming language.\n",
    "It supports object-oriented programming.\n",
    "File handling is done using the open() function.\n",
    "Files can be read, written and appended.\n",
    "The with statement closes a file automatically.\n",
]
with open("input.txt", "w") as f:          # create sample input file
    f.writelines(lines)

try:
    with open("input.txt", "r") as f:
        data = f.readlines()
    print("Contents of input.txt:")
    for n, line in enumerate(data, 1):
        print(f"  {n}: {line.rstrip()}")

    print("\nTotal number of lines:", len(data))
    first_two = data[:2]
    print("First two lines:")
    for line in first_two:
        print("  ", line.rstrip())

    with open("output.txt", "w") as out:
        out.writelines(first_two)
    print("\nExtracted lines written to output.txt")

    with open("output.txt") as out:
        print("Contents of output.txt:")
        print(out.read())
except FileNotFoundError:
    print("Input file not found!")

try:
    open("missing.txt")
except FileNotFoundError as e:
    print("Exception handled ->", e)
```

Output:

```
Contents of input.txt:
  1: Python is a high-level programming language.
  2: It supports object-oriented programming.
  3: File handling is done using the open() function.
  4: Files can be read, written and appended.
  5: The with statement closes a file automatically.
Total number of lines: 5
First two lines:
   Python is a high-level programming language.
   It supports object-oriented programming.
Extracted lines written to output.txt
Contents of output.txt:
Python is a high-level programming language.
It supports object-oriented programming.
Exception handled -> [Errno 2] No such file or directory: 'missing.txt'
```

Experiment No. 9

Code:

```python
# Experiment 9: Read CSV and convert to JSON
import csv, json

with open("students.csv", "w", newline="") as f:      # sample CSV input
    w = csv.writer(f)
    w.writerow(["id", "name", "branch", "cgpa"])
    w.writerows([[1, "Aarav", "CSE", 8.9], [2, "Diya", "IT", 9.1],
                 [3, "Kabir", "CSE", 7.8], [4, "Meera", "ECE", 8.4]])

print("students.csv:")
print(open("students.csv").read())

data = []
with open("students.csv", newline="") as f:
    for row in csv.DictReader(f):
        row["id"] = int(row["id"])
        row["cgpa"] = float(row["cgpa"])
        data.append(row)

with open("students.json", "w") as f:
    json.dump(data, f, indent=4)

print("students.json:")
print(open("students.json").read())

with open("students.json") as f:                       # verify by reading back
    loaded = json.load(f)
print("Records converted :", len(loaded))
print("Top scorer        :", max(loaded, key=lambda r: r["cgpa"])["name"])
```

Output:

```
students.csv:
id,name,branch,cgpa
1,Aarav,CSE,8.9
2,Diya,IT,9.1
3,Kabir,CSE,7.8
4,Meera,ECE,8.4
students.json:
[
    {
        "id": 1,
        "name": "Aarav",
        "branch": "CSE",
        "cgpa": 8.9
    },
    {
        "id": 2,
        "name": "Diya",
        "branch": "IT",
        "cgpa": 9.1
    },
    {
        "id": 3,
        "name": "Kabir",
        "branch": "CSE",
        "cgpa": 7.8
    },
    {
        "id": 4,
        "name": "Meera",
        "branch": "ECE",
        "cgpa": 8.4
    }
]
Records converted : 4
Top scorer        : Diya
```

Experiment No. 10

Code:

```python
# Experiment 10: NumPy - 1-D array, slicing, statistics, broadcasting
import numpy as np

arr = np.arange(1, 11)
print("Array            :", arr)
print("Shape, dtype     :", arr.shape, arr.dtype)

print("\n--- Indexing and Slicing ---")
print("arr[0]           :", arr[0])
print("arr[-1]          :", arr[-1])
print("arr[2:6]         :", arr[2:6])
print("arr[:4]          :", arr[:4])
print("arr[5:]          :", arr[5:])
print("arr[::2]         :", arr[::2])
print("arr[::-1]        :", arr[::-1])
print("arr[arr % 2 == 0]:", arr[arr % 2 == 0])

print("\n--- Statistical measures ---")
print("Sum              :", arr.sum())
print("Mean             :", arr.mean())
print("Maximum          :", arr.max())
print("Minimum          :", arr.min())
print("Std deviation    :", round(arr.std(), 4))
print("Cumulative sum   :", arr.cumsum())

print("\n--- Broadcasting ---")
print("arr + 5          :", arr + 5)
print("arr * 2          :", arr * 2)
print("arr ** 2         :", arr ** 2)
print("arr / 10         :", arr / 10)
col = np.array([[1], [2], [3]])
print("\n(3x1) + (10,) broadcast to 3x10:")
print(col + arr)

mod = arr.copy()
mod[mod > 5] *= 10
print("\nElements > 5 multiplied by 10:", mod)
```

Output:

```
Array            : [ 1  2  3  4  5  6  7  8  9 10]
Shape, dtype     : (10,) int64
--- Indexing and Slicing ---
arr[0]           : 1
arr[-1]          : 10
arr[2:6]         : [3 4 5 6]
arr[:4]          : [1 2 3 4]
arr[5:]          : [ 6  7  8  9 10]
arr[::2]         : [1 3 5 7 9]
arr[::-1]        : [10  9  8  7  6  5  4  3  2  1]
arr[arr % 2 == 0]: [ 2  4  6  8 10]
--- Statistical measures ---
Sum              : 55
Mean             : 5.5
Maximum          : 10
Minimum          : 1
Std deviation    : 2.8723
Cumulative sum   : [ 1  3  6 10 15 21 28 36 45 55]
--- Broadcasting ---
arr + 5          : [ 6  7  8  9 10 11 12 13 14 15]
arr * 2          : [ 2  4  6  8 10 12 14 16 18 20]
arr ** 2         : [  1   4   9  16  25  36  49  64  81 100]
arr / 10         : [0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. ]
(3x1) + (10,) broadcast to 3x10:
[[ 2  3  4  5  6  7  8  9 10 11]
 [ 3  4  5  6  7  8  9 10 11 12]
 [ 4  5  6  7  8  9 10 11 12 13]]
Elements > 5 multiplied by 10: [  1   2   3   4   5  60  70  80  90 100]
```

Experiment No. 11

Code:

```python
# Experiment 11: Pandas Series with ten random numbers
import numpy as np
import pandas as pd

np.random.seed(42)                                  # reproducible output
s = pd.Series(np.random.randint(10, 100, 10), name="Random")
print("Series with 10 random numbers:")
print(s)

print("\n--- Indexing ---")
print("s[0]       :", s[0])
print("s[2:5]     :\n" + s[2:5].to_string())
s.index = list("ABCDEFGHIJ")
print("\nCustom index, s['C'] :", s["C"])
print("s.loc[['A','J']]:\n" + s.loc[["A", "J"]].to_string())

print("\n--- Filtering (values > 50) ---")
print(s[s > 50])

print("\n--- Statistical operations ---")
print("Mean    :", s.mean())
print("Median  :", s.median())
print("Minimum :", s.min(), " at index", s.idxmin())
print("Maximum :", s.max(), " at index", s.idxmax())
print("Sum     :", s.sum())
print("Std dev :", round(s.std(), 3))

print("\n--- describe() ---")
print(s.describe())

print("\nSorted descending:")
print(s.sort_values(ascending=False).head(3))
```

Output:

```
Series with 10 random numbers:
0    61
1    24
2    81
3    70
4    30
5    92
6    96
7    84
8    84
9    97
Name: Random, dtype: int64
--- Indexing ---
s[0]       : 61
s[2:5]     :
2    81
3    70
4    30
Custom index, s['C'] : 81
s.loc[['A','J']]:
A    61
J    97
--- Filtering (values > 50) ---
A    61
C    81
D    70
F    92
G    96
H    84
I    84
J    97
Name: Random, dtype: int64
--- Statistical operations ---
Mean    : 71.9
Median  : 82.5
Minimum : 24  at index B
Maximum : 97  at index J
Sum     : 719
Std dev : 26.168
--- describe() ---
```

```
count    10.000000
mean     71.900000
std      26.168047
min      24.000000
25%      63.250000
50%      82.500000
75%      90.000000
max      97.000000
Name: Random, dtype: float64
Sorted descending:
J    97
G    96
F    92
Name: Random, dtype: int64
```

Experiment No. 12

Code:

```python
# Experiment 12: Exploratory Data Analysis on a customer churn dataset
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", 20)

# ---------- 1. Create a sample customer churn dataset (with missing values and outliers) ----------
rng = np.random.default_rng(7)
n = 500
tenure = rng.integers(1, 72, n)
contract = rng.choice(["Month-to-month", "One year", "Two year"], n, p=[0.55, 0.25, 0.20])
monthly = np.round(rng.normal(65, 20, n).clip(20, 120), 2)
total = np.round(monthly * tenure + rng.normal(0, 50, n), 2)
support_calls = rng.poisson(2, n)
p_churn = 0.10 + 0.30 * (contract == "Month-to-month") + 0.04 * support_calls - 0.002 * tenure
churn = np.where(rng.random(n) < p_churn.clip(0.02, 0.9), "Yes", "No")
df = pd.DataFrame({"CustomerID": np.arange(1001, 1001 + n),
                   "Gender": rng.choice(["Male", "Female"], n),
                   "Tenure": tenure, "Contract": contract,
                   "MonthlyCharges": monthly, "TotalCharges": total,
                   "SupportCalls": support_calls, "Churn": churn})
df.loc[rng.choice(n, 25, replace=False), "MonthlyCharges"] = np.nan   # inject missing values
df.loc[rng.choice(n, 15, replace=False), "TotalCharges"] = np.nan
df.loc[rng.choice(n, 10, replace=False), "Contract"] = np.nan
df.loc[[5, 50, 120], "MonthlyCharges"] = [480.0, 520.0, 610.0]       # inject outliers
df.to_csv("customer_churn.csv", index=False)

# ---------- 2. Understand the data ----------
print("Shape:", df.shape)
print("\nFirst 5 rows:\n", df.head())
print("\nData types:\n", df.dtypes)

# ---------- 3. Missing values ----------
print("\nMissing values per column:\n", df.isnull().sum())
df["MonthlyCharges"] = df["MonthlyCharges"].fillna(df["MonthlyCharges"].median())
df["TotalCharges"] = df["TotalCharges"].fillna(df["TotalCharges"].median())
df["Contract"] = df["Contract"].fillna(df["Contract"].mode()[0])
print("\nMissing values after cleaning:", int(df.isnull().sum().sum()))
print("Duplicate rows:", int(df.duplicated().sum()))

# ---------- 4. Descriptive statistics ----------
print("\nDescriptive statistics:\n", df.describe().round(2))
print("\nMedian MonthlyCharges:", df["MonthlyCharges"].median())
print("Mode of Contract     :", df["Contract"].mode()[0])
print("Variance of Tenure   :", round(df["Tenure"].var(), 2))
print("\nChurn counts:\n", df["Churn"].value_counts())
print("\nChurn rate (%):\n", (df["Churn"].value_counts(normalize=True) * 100).round(2))

# ---------- 5. Outliers (IQR method) ----------
Q1, Q3 = df["MonthlyCharges"].quantile([0.25, 0.75])
IQR = Q3 - Q1
low, high = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
outliers = df[(df["MonthlyCharges"] < low) | (df["MonthlyCharges"] > high)]
print(f"\nIQR bounds for MonthlyCharges: {low:.2f} to {high:.2f}")
print("Outliers found:", len(outliers))
print(outliers[["CustomerID", "MonthlyCharges"]])
df["MonthlyCharges"] = df["MonthlyCharges"].clip(low, high)           # cap outliers
print("Max MonthlyCharges after capping:", df["MonthlyCharges"].max())

# ---------- 6. Patterns ----------
print("\nChurn rate by Contract (%):\n",
      (df.groupby("Contract")["Churn"].apply(lambda x: (x == "Yes").mean() * 100)).round(2))
print("\nAverage values by Churn:\n",
      df.groupby("Churn")[["Tenure", "MonthlyCharges", "SupportCalls"]].mean().round(2))
corr = df[["Tenure", "MonthlyCharges", "TotalCharges", "SupportCalls"]].corr().round(2)
print("\nCorrelation matrix:\n", corr)

# ---------- 7. Visualisation ----------
fig, ax = plt.subplots(2, 3, figsize=(15, 8))
ax[0, 0].hist(df["Tenure"], bins=15, edgecolor="black"); ax[0, 0].set_title("Tenure Distribution")
sns.boxplot(y=df["MonthlyCharges"], ax=ax[0, 1]); ax[0, 1].set_title("Box Plot: MonthlyCharges (capped)")
```

```python
sns.countplot(x="Churn", data=df, ax=ax[0, 2]); ax[0, 2].set_title("Churn Count")
sns.countplot(x="Contract", hue="Churn", data=df, ax=ax[1, 0]); ax[1, 0].set_title("Churn by Contract")
ax[1, 0].tick_params(axis="x", labelsize=8)
sns.scatterplot(x="Tenure", y="TotalCharges", hue="Churn", data=df, ax=ax[1, 1], s=20)
ax[1, 1].set_title("Tenure vs TotalCharges")
sns.heatmap(corr, annot=True, cmap="coolwarm", ax=ax[1, 2]); ax[1, 2].set_title("Correlation Matrix")
plt.tight_layout()
plt.savefig("eda_plots.png", dpi=110)
print("\nPlots saved to eda_plots.png")
```

Output:

```
Shape: (500, 8)
First 5 rows:
    CustomerID  Gender  Tenure        Contract  MonthlyCharges  TotalCharges  SupportCalls Churn
0        1001    Male      68  Month-to-month           34.79       2398.79             4    No
1        1002    Male      45  Month-to-month           70.95       3116.20             3    No
2        1003  Female      49  Month-to-month           62.81       3071.14             0    No
3        1004    Male      64  Month-to-month           58.73       3774.42             2    No
4        1005    Male      42  Month-to-month           63.54       2649.52             3   Yes
Data types:
 CustomerID          int64
Gender                str
Tenure              int64
Contract              str
MonthlyCharges    float64
TotalCharges      float64
SupportCalls        int64
Churn                 str
dtype: object
Missing values per column:
 CustomerID         0
Gender              0
Tenure              0
Contract           10
MonthlyCharges     25
TotalCharges       15
SupportCalls        0
Churn               0
dtype: int64
Missing values after cleaning: 0
Duplicate rows: 0
Descriptive statistics:
        CustomerID  Tenure  MonthlyCharges  TotalCharges  SupportCalls
count       500.00  500.00          500.00        500.00        500.00
mean       1250.50   37.18           67.72       2402.85          2.08
std         144.48   20.65           41.21       1540.25          1.46
min        1001.00    1.00           20.00         32.99          0.00
25%        1125.75   18.75           52.66       1128.92          1.00
50%        1250.50   38.00           63.88       2275.32          2.00
75%        1375.25   55.25           77.85       3375.69          3.00
max        1500.00   71.00          610.00       7784.72          6.00
Median MonthlyCharges: 63.88
Mode of Contract     : Month-to-month
Variance of Tenure   : 426.49
Churn counts:
 Churn
No     363
Yes    137
Name: count, dtype: int64
Churn rate (%):
 Churn
No     72.6
Yes    27.4
Name: proportion, dtype: float64
IQR bounds for MonthlyCharges: 14.89 to 115.62
Outliers found: 4
     CustomerID  MonthlyCharges
5          1006          480.00
50         1051          520.00
93         1094          116.43
120        1121          610.00
Max MonthlyCharges after capping: 115.62499999999999
Churn rate by Contract (%):
 Contract
Month-to-month    40.86
One year           9.16
Two year          12.22
Name: Churn, dtype: float64
```

```
Average values by Churn:
       Tenure  MonthlyCharges  SupportCalls
Churn
No      38.26           65.38          1.91
Yes     34.32           64.70          2.52
Correlation matrix:
                Tenure  MonthlyCharges  TotalCharges  SupportCalls
Tenure            1.00           -0.00          0.82          0.02
MonthlyCharges   -0.00            1.00          0.44          0.01
TotalCharges      0.82            0.44          1.00          0.04
SupportCalls      0.02            0.01          0.04          1.00
Plots saved to eda_plots.png
```











